

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
“КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО”

Факультет програмних систем та прикладної математики
Кафедра програмного забезпечення комп’ютерних систем



Лабораторна робота № 1

з дисципліни “Розроблення вебзастосунків ”

тема «««Проєктування REST API та його реалізація »

Виконав
студент III курсу
групи КП- 42
Жижка Богдан

Київ 2026

Мета роботи: Отримати навички проєктування та реалізації REST API вебзастосунку, що буде надавати доступ до декількох сутностей обраної предметної галузі.

Завдання:

1. Студент самостійно обирає предметну галузь, котрій буде присвячена робота його комп'ютерного практикуму.
2. В рамках обраної чи наданої предметної галузі, студент самостійно ідентифікує щонайменше дві сутності та їх властивості.
3. Для ідентифікованих сутностей, студент проєктує та реалізовує веб-API, що повинно повністю відповідати принципам REST, та яке надає визначені студентом операції для роботи з цими сутностями.
4. Спроєктоване та реалізоване REST API повинно обов'язково надавати наступні операції для кожної сутності: a. отримання списку всіх сутностей; b. отримання певної сутності за її ідентифікатором; c. додавання нової сутності; d. оновлення існуючої сутності; e. видалення всіх сутностей; f. видалення певної сутності за її ідентифікатором.
5. Якщо ідентифіковані студентом сутності пов'язані між собою зв'язком «один-до-багатьох» або «багато-до-багатьох», тоді сутності також повинні мати операції для отримання пов'язаних сутностей.

ХІД РОБОТИ

1. Постановка задачі Для виконання комп'ютерного практикуму було обрано предметну галузь «Система управління таксопарком». Дана система призначена для обліку автомобілів таксопарку, а також фіксації водіїв, що за ними закріплені. Ця предметна галузь дозволяє ефективно реалізувати базові принципи REST API для керування ресурсами.

У межах обраної предметної галузі було спроєктовано та реалізовано дві основні сутності, які мають між собою логічний зв'язок «один-до-багатьох» (один автомобіль може мати багато водіїв, що працюють позмінно, але кожен водій в даний момент часу закріплений за одним авто).

Автомобіль (Car) — сутність, що репрезентує транспортний засіб у системі. Містить наступні властивості:

- Id (ціле число) — унікальний ідентифікатор автомобіля в базі даних.
- Brand (рядок) — марка автомобіля.
- Model (рядок) — модель автомобіля.
- EngineVolume (дійсне число) — об'єм двигуна.

- Year (ціле число) — рік випуску транспортного засобу.
- Age (ціле число) — вік автомобіля (обчислюване поле, яке генерується на основі поточного року та року випуску).

Водій (Driver) — сутність, що зберігає дані про працівника таксопарку. Містить наступні властивості:

- Id (ціле число) — унікальний ідентифікатор запису про водія.
- FullName (рядок) — повне ім'я водія.
- Rating (дійсне число) — рейтинг водія в системі оцінювання.
- IsBusy (логічне значення) — поточний статус водія (вільний або виконує замовлення).
- CarId (ціле число) — ідентифікатор автомобіля, за яким закріплений водій. Логічно посилається на сутність Car та є обов'язковим для створення запису.

OpenAPI специфікація реалізованого веб-API.


```
{
 "openapi": "3.0.1",
 "info": {
  "title": "Taxi",
  "version": "1.0"
 },
 "paths": {
  "/api/Cars": {
   "get": {
    "tags": [
     "Cars"
    ],
    "responses": {
     "200": {
      "description": "OK",
      "content": {
       "text/plain": {
        "schema": {
         "type": "array",
```

```
          "items": {
            "$ref": "#/components/schemas/Car"
          }
        }
      },
      "application/json": {
        "schema": {
          "type": "array",
          "items": {
            "$ref": "#/components/schemas/Car"
          }
        }
      },
      "text/json": {
        "schema": {
          "type": "array",
          "items": {
            "$ref": "#/components/schemas/Car"
          }
        }
      }
    }
  }
},
"post": {
  "tags": [
    "Cars"
  ],
  "requestBody": {
    "content": {
      "application/json": {
        "schema": {
          "$ref": "#/components/schemas/Car"
        }
```

```
    },
    "text/json": {
      "schema": {
        "$ref": "#/components/schemas/Car"
      }
    },
    "application/*+json": {
      "schema": {
        "$ref": "#/components/schemas/Car"
      }
    }
  }
},
"responses": {
  "200": {
    "description": "OK",
    "content": {
      "text/plain": {
        "schema": {
          "$ref": "#/components/schemas/Car"
        }
      },
      "application/json": {
        "schema": {
          "$ref": "#/components/schemas/Car"
        }
      },
      "text/json": {
        "schema": {
          "$ref": "#/components/schemas/Car"
        }
      }
    }
  }
}
```

```
  },
  "delete": {
    "tags": [
      "Cars"
    ],
    "responses": {
      "200": {
        "description": "OK"
      }
    }
  }
},
"/api/Cars/{id}": {
  "get": {
    "tags": [
      "Cars"
    ],
    "parameters": [
      {
        "name": "id",
        "in": "path",
        "required": true,
        "schema": {
          "type": "integer",
          "format": "int32"
        }
      }
    ],
    "responses": {
      "200": {
        "description": "OK",
        "content": {
          "text/plain": {
            "schema": {
              "$ref": "#/components/schemas/Car"
```

```
        }
       },
       "application/json": {
        "schema": {
         "$ref": "#/components/schemas/Car"
        }
       },
       "text/json": {
        "schema": {
         "$ref": "#/components/schemas/Car"
        }
       }
      }
     }
    }
   },
   "put": {
    "tags": [
     "Cars"
    ],
    "parameters": [
     {
      "name": "id",
      "in": "path",
      "required": true,
      "schema": {
       "type": "integer",
       "format": "int32"
      }
     }
    ],
    "requestBody": {
     "content": {
      "application/json": {
       "schema": {
```

```
        "$ref": "#/components/schemas/Car"
      }
    },
    "text/json": {
      "schema": {
        "$ref": "#/components/schemas/Car"
      }
    },
    "application/*+json": {
      "schema": {
        "$ref": "#/components/schemas/Car"
      }
    }
  }
},
"responses": {
  "200": {
    "description": "OK",
    "content": {
      "text/plain": {
        "schema": {
          "$ref": "#/components/schemas/Car"
        }
      },
      "application/json": {
        "schema": {
          "$ref": "#/components/schemas/Car"
        }
      },
      "text/json": {
        "schema": {
          "$ref": "#/components/schemas/Car"
        }
      }
    }
```

```
      }
    }
  },
  "delete": {
    "tags": [
      "Cars"
    ],
    "parameters": [
      {
        "name": "id",
        "in": "path",
        "required": true,
        "schema": {
          "type": "integer",
          "format": "int32"
        }
      }
    ],
    "responses": {
      "200": {
        "description": "OK"
      }
    }
  }
},
"/api/Cars/{id}/drivers": {
  "get": {
    "tags": [
      "Cars"
    ],
    "parameters": [
      {
        "name": "id",
        "in": "path",
        "required": true,
```

```
    "schema": {
     "type": "integer",
     "format": "int32"
    }
   }
],
"responses": {
  "200": {
    "description": "OK",
    "content": {
      "text/plain": {
        "schema": {
          "type": "array",
          "items": {
            "$ref": "#/components/schemas/Driver"
          }
        }
      },
      "application/json": {
        "schema": {
          "type": "array",
          "items": {
            "$ref": "#/components/schemas/Driver"
          }
        }
      },
      "text/json": {
        "schema": {
          "type": "array",
          "items": {
            "$ref": "#/components/schemas/Driver"
          }
        }
      }
    }
```

```
      }
    }
  }
},
"/api/Drivers": {
  "get": {
    "tags": [
      "Drivers"
    ],
    "responses": {
      "200": {
        "description": "OK",
        "content": {
          "text/plain": {
            "schema": {
              "type": "array",
              "items": {
                "$ref": "#/components/schemas/Driver"
              }
            }
          },
          "application/json": {
            "schema": {
              "type": "array",
              "items": {
                "$ref": "#/components/schemas/Driver"
              }
            }
          },
          "text/json": {
            "schema": {
              "type": "array",
              "items": {
                "$ref": "#/components/schemas/Driver"
              }
```

```
        }
       }
      }
     }
    }
   },
   "post": {
    "tags": [
     "Drivers"
    ],
    "requestBody": {
     "content": {
      "application/json": {
       "schema": {
        "$ref": "#/components/schemas/Driver"
       }
      },
      "text/json": {
       "schema": {
        "$ref": "#/components/schemas/Driver"
       }
      },
      "application/*+json": {
       "schema": {
        "$ref": "#/components/schemas/Driver"
       }
      }
     }
    },
    "responses": {
     "200": {
      "description": "OK",
      "content": {
       "text/plain": {
        "schema": {
```

```
            "$ref": "#/components/schemas/Driver"
          }
        },
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/Driver"
          }
        },
        "text/json": {
          "schema": {
            "$ref": "#/components/schemas/Driver"
          }
        }
      }
    }
  },
  "delete": {
    "tags": [
      "Drivers"
    ],
    "responses": {
      "200": {
        "description": "OK"
      }
    }
  }
},
"/api/Drivers/{id}": {
  "get": {
    "tags": [
      "Drivers"
    ],
    "parameters": [
      {
```

```
      "name": "id",
      "in": "path",
      "required": true,
      "schema": {
        "type": "integer",
        "format": "int32"
      }
    }
  ],
  "responses": {
    "200": {
      "description": "OK",
      "content": {
        "text/plain": {
          "schema": {
            "$ref": "#/components/schemas/Driver"
          }
        },
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/Driver"
          }
        },
        "text/json": {
          "schema": {
            "$ref": "#/components/schemas/Driver"
          }
        }
      }
    }
  }
},
"put": {
  "tags": [
    "Drivers"
```

```
],
"parameters": [
  {
    "name": "id",
    "in": "path",
    "required": true,
    "schema": {
      "type": "integer",
      "format": "int32"
    }
  }
],
"requestBody": {
  "content": {
    "application/json": {
      "schema": {
        "$ref": "#/components/schemas/Driver"
      }
    },
    "text/json": {
      "schema": {
        "$ref": "#/components/schemas/Driver"
      }
    },
    "application/*+json": {
      "schema": {
        "$ref": "#/components/schemas/Driver"
      }
    }
  }
},
"responses": {
  "200": {
    "description": "OK",
    "content": {
```

```
        "text/plain": {
          "schema": {
            "$ref": "#/components/schemas/Driver"
          }
        },
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/Driver"
          }
        },
        "text/json": {
          "schema": {
            "$ref": "#/components/schemas/Driver"
          }
        }
      }
    }
  }
},
"delete": {
  "tags": [
    "Drivers"
  ],
  "parameters": [
    {
      "name": "id",
      "in": "path",
      "required": true,
      "schema": {
        "type": "integer",
        "format": "int32"
      }
    }
  ],
  "responses": {
```

```
      "200": {
        "description": "OK"
      }
    }
  }
 }
},
"components": {
 "schemas": {
  "Car": {
   "type": "object",
   "properties": {
    "id": {
     "type": "integer",
     "format": "int32"
    },
    "brand": {
     "type": "string",
     "nullable": true
    },
    "model": {
     "type": "string",
     "nullable": true
    },
    "engineVolume": {
     "type": "number",
     "format": "double"
    },
    "year": {
     "type": "integer",
     "format": "int32"
    },
    "age": {
     "type": "integer",
     "format": "int32",
```

```
          "readOnly": true
        }
      },
      "additionalProperties": false
    },
    "Driver": {
      "type": "object",
      "properties": {
        "id": {
          "type": "integer",
          "format": "int32"
        },
        "fullName": {
          "type": "string",
          "nullable": true
        },
        "rating": {
          "type": "number",
          "format": "double"
        },
        "isBusy": {
          "type": "boolean"
        },
        "carId": {
          "type": "integer",
          "format": "int32"
        }
      },
      "additionalProperties": false
    }
  }
}
```

Код розробленого програмного забезпечення

Controllers

```
using Microsoft.AspNetCore.Mvc;
using System.Collections.Generic;
using Taxi.Models;
using Taxi.Services;

namespace Taxi.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class CarsController : ControllerBase
    {
        private readonly ICarService carService;

        public CarsController(ICarService carService)
        {
            this.carService = carService;
        }

        [HttpGet]
        public ActionResult<List<Car>> GetAll()
        {
            var cars = carService.GetAll();
            return Ok(cars);
        }

        [HttpGet("{id}")]
        public ActionResult<Car> GetById(int id)
        {
            var car = carService.GetById(id);
            return Ok(car);
        }

        [HttpPost]
        public ActionResult<Car> Add([FromBody] Car car)
```

```
        {
            var newCar = carService.Add(car);
                return CreatedAtAction(nameof(GetById), new { id =
newCar.Id }, newCar);
        }

        [HttpPut("{id}")]
        public ActionResult<Car> Update(int id, [FromBody] Car car)
        {
            var updatedCar = carService.Update(id, car);
            return Ok(updatedCar);
        }

        [HttpDelete("{id}")]
        public ActionResult Delete(int id)
        {
            carService.Delete(id);
            return NoContent();
        }

        [HttpDelete]
        public ActionResult DeleteAll()
        {
            carService.DeleteAll();
            return NoContent();
        }

        [HttpGet("{id}/drivers")]
        public ActionResult<List<Driver>> GetDriversByCarId(int id)
        {
            var drivers = carService.GetDriversByCarId(id);
            return Ok(drivers);
        }
    }
}
```

```
using Microsoft.AspNetCore.Mvc;
using System.Collections.Generic;
using Taxi.Models;
using Taxi.Services;

namespace Taxi.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class DriversController : ControllerBase
    {
        private readonly IDriverService driverService;

        public DriversController(IDriverService driverService)
        {
            this.driverService = driverService;
        }

        [HttpGet]
        public ActionResult<List<Driver>> GetAll()
        {
            var drivers = driverService.GetAll();
            return Ok(drivers);
        }

        [HttpGet("{id}")]
        public ActionResult<Driver> GetById(int id)
        {
            var driver = driverService.GetById(id);
            return Ok(driver);
        }

        [HttpPost]
        public ActionResult<Driver> Add([FromBody] Driver driver)
        {
```

```
            var newDriver = driverService.Add(driver);
                return CreatedAtAction(nameof(GetById), new { id =
newDriver.Id }, newDriver);
        }

        [HttpPut("{id}")]
            public ActionResult<Driver> Update(int id, [FromBody]
Driver driver)
        {
            var updatedDriver = driverService.Update(id, driver);
            return Ok(updatedDriver);
        }

        [HttpDelete("{id}")]
        public ActionResult Delete(int id)
        {
            driverService.Delete(id);
            return NoContent();
        }

        [HttpDelete]
        public ActionResult DeleteAll()
        {
            driverService.DeleteAll();
            return NoContent();
        }
    }
}
```

Exception

```
using System;

namespace Taxi.Exceptions
{
    public class BadRequestException : Exception
    {
```

```
        public BadRequestException(string message) : base(message)
        {
        }
    }
}
```

```
using System;

namespace Taxi.Exceptions
{
    public class NotFoundException : Exception
    {
        public NotFoundException(string message) : base(message)
        {
        }
    }
}
```

Middleware

```
using Microsoft.AspNetCore.Http;
using System;
using System.Text.Json;
using System.Threading.Tasks;
using Taxi.Exceptions;

namespace Taxi.Middleware
{
    public class ExceptionHandlingMiddleware
    {
        private readonly RequestDelegate next;

        public ExceptionHandlingMiddleware(RequestDelegate next)
        {
            this.next = next;
        }
```

```
        public async Task InvokeAsync(HttpContext context)
        {
            try
            {
                await next(context);
            }
            catch (Exception ex)
            {
                await HandleExceptionAsync(context, ex);
            }
        }

              private  static  Task  HandleExceptionAsync(HttpContext
context, Exception exception)
        {
            context.Response.ContentType = "application/json";
                                        context.Response.StatusCode    =
StatusCodes.Status500InternalServerError;
            var message = "Внутрішня помилка сервера.";

            if (exception is NotFoundException)
            {
                                         context.Response.StatusCode   =
StatusCodes.Status404NotFound;
                message = exception.Message;
            }
            else if (exception is BadRequestException)
            {
                                        context.Response.StatusCode   =
StatusCodes.Status400BadRequest;
                message = exception.Message;
            }

               var  result  =  JsonSerializer.Serialize(new  {  error  =
message,  status  =  context.Response.StatusCode  });
            return context.Response.WriteAsync(result);
```

```
        }
    }
}
```

Models

```
using System.ComponentModel.DataAnnotations;

namespace Taxi.Models
{
    public class Car
    {
        public int Id { get; set; }
        public string Brand { get; set; } = string.Empty;
        public string Model { get; set; } = string.Empty;
        public double EngineVolume { get; set; }
        public int Year { get; set; }
        public int Age => DateTime.Now.Year - Year;
    }
}
```

```
using System.Collections.Generic;

namespace Taxi.Models
{
    public class DataStore
    {
        public List<Car> Cars { get; set; } = new List<Car>();
              public  List<Driver>  Driver  {  get;  set;  }  =  new
List<Driver>();
    }
}
```

```
using System.ComponentModel.DataAnnotations;

namespace Taxi.Models;
```

```
public class Driver
{
    public int Id { get; set; }
    public string FullName { get; set; } = string.Empty;
    public double Rating { get; set; }
    public bool IsBusy { get; set; }
    public int CarId { get; set; }
}
```

Services

```
using System.Collections.Generic;
using System.Linq;
using Microsoft.Extensions.Logging;
using Taxi.Exceptions;
using Taxi.Models;

namespace Taxi.Services
{
    public class CarService : ICarService
    {
        private readonly IJsonDataService dataService;
        private readonly ILogger<CarService> logger;

                    public  CarService(IJsonDataService  dataService,
ILogger<CarService> logger)
        {
            this.dataService = dataService;
            this.logger = logger;
        }

        public List<Car> GetAll()
        {
             logger.LogInformation("Виконано запит на отримання всіх
автомобілів.");
            var data = dataService.LoadData();
            return data.Cars;
```

```
        }

        public Car GetById(int id)
        {
            logger.LogInformation($"Пошук автомобіля з ID: {id}");
            var data = dataService.LoadData();
            var car = data.Cars.FirstOrDefault(c => c.Id == id);

            if (car == null)
            {
                 logger.LogWarning($"[Увага] Автомобіль з ID {id} не
знайдено в базі!");
                  throw new NotFoundException($"Автомобіль з ID {id}
не знайдено.");
            }

                        logger.LogInformation($"Автомобіль  знайдено:
{car.Brand} {car.Model}");
            return car;
        }

        public Car Add(Car car)
        {
            var data = dataService.LoadData();

            if (data.Cars.Any())
            {
                car.Id = data.Cars.Max(c => c.Id) + 1;
            }
            else
            {
                car.Id = 1;
            }

            data.Cars.Add(car);
            dataService.SaveData(data);
```

```
                    logger.LogInformation($"[Успіх] Додано новий
автомобіль: {car.Brand} {car.Model}, ID: {car.Id}");
            return car;
        }

        public Car Update(int id, Car updatedCar)
        {
            var data = dataService.LoadData();
            var car = data.Cars.FirstOrDefault(c => c.Id == id);

            if (car == null)
            {
                        logger.LogWarning($"[Увага] Спроба оновити
неіснуючий автомобіль (ID {id})");
                  throw new NotFoundException($"Автомобіль з ID {id}
не знайдено.");
            }

            car.Brand = updatedCar.Brand;
            car.Model = updatedCar.Model;
            car.EngineVolume = updatedCar.EngineVolume;
            car.Year = updatedCar.Year;

            dataService.SaveData(data);
                 logger.LogInformation($"[Успіх] Автомобіль ID {id}
успішно оновлено.");
            return car;
        }

        public void Delete(int id)
        {
            var data = dataService.LoadData();
            var car = data.Cars.FirstOrDefault(c => c.Id == id);

            if (car == null)
```

```
            {
                        logger.LogWarning($"[Увага] Спроба видалити
неіснуючий автомобіль (ID {id})");
                throw new NotFoundException($"Автомобіль з ID {id}
не знайдено.");
            }

            data.Cars.Remove(car);
            data.Driver.RemoveAll(d => d.CarId == id);

            dataService.SaveData(data);
              logger.LogInformation($"[Успіх] Автомобіль ID {id} та
його водіїв видалено з бази.");
        }

        public void DeleteAll()
        {
            var data = dataService.LoadData();
            data.Cars.Clear();
            data.Driver.Clear();
            dataService.SaveData(data);
             logger.LogInformation("[Увага] Всі автомобілі та водії
були видалені з бази даних!");
        }

        public List<Driver> GetDriversByCarId(int carId)
        {
            var data = dataService.LoadData();

            if (!data.Cars.Any(c => c.Id == carId))
            {
                   logger.LogWarning($"[Увага] Спроба знайти водіїв
для неіснуючого авто (ID {carId})");
                     throw new NotFoundException($"Автомобіль з ID
{carId} не знайдено.");
            }
```

```
                    return  data.Driver.Where(d  =>  d.CarId  ==
carId).ToList();
        }
    }
}
```

```
using System.Collections.Generic;
using System.Linq;
using Microsoft.Extensions.Logging;
using Taxi.Exceptions;
using Taxi.Models;

namespace Taxi.Services
{
    public class DriverService : IDriverService
    {
        private readonly IJsonDataService dataService;
        private readonly ILogger<DriverService> logger;

                public  DriverService(IJsonDataService  dataService,
ILogger<DriverService> logger)
        {
            this.dataService = dataService;
            this.logger = logger;
        }

        public List<Driver> GetAll()
        {
             logger.LogInformation("Виконано запит на отримання всіх
водіїв.");
            var data = dataService.LoadData();
            return data.Driver;
        }

        public Driver GetById(int id)
```

```
        {
            logger.LogInformation($"Пошук водія з ID: {id}");
            var data = dataService.LoadData();
               var driver = data.Driver.FirstOrDefault(d => d.Id ==
id);

            if (driver == null)
            {
                     logger.LogWarning($"[Увага] Водія з ID {id} не
знайдено в базі!");
                    throw new NotFoundException($"Водія з ID {id} не
знайдено.");
            }

                            logger.LogInformation($"Водія  знайдено:
{driver.FullName}");
            return driver;
        }

        public Driver Add(Driver driver)
        {
            var data = dataService.LoadData();

            if (!data.Cars.Any(c => c.Id == driver.CarId))
            {
                     logger.LogWarning($"[Помилка] Спроба прив'язати
водія до неіснуючого авто (CarId {driver.CarId})");
                    throw new BadRequestException($"Неможливо додати
водія: автомобіль з ID {driver.CarId} не існує.");
            }

            if (data.Driver.Any())
            {
                driver.Id = data.Driver.Max(d => d.Id) + 1;
            }
            else
```

```
            {
                driver.Id = 1;
            }

            data.Driver.Add(driver);
            dataService.SaveData(data);

                logger.LogInformation($"[Успіх] Додано нового водія:
{driver.FullName}, ID: {driver.Id}");
            return driver;
        }

        public Driver Update(int id, Driver updatedDriver)
        {
            var data = dataService.LoadData();
               var driver = data.Driver.FirstOrDefault(d => d.Id ==
id);

            if (driver == null)
            {
                        logger.LogWarning($"[Увага] Спроба оновити
неіснуючого водія (ID {id})");
                    throw new NotFoundException($"Водія з ID {id} не
знайдено.");
            }

                    if (driver.CarId != updatedDriver.CarId &&
!data.Cars.Any(c => c.Id == updatedDriver.CarId))
            {
                    logger.LogWarning($"[Помилка] Спроба перевести
водія на неіснуюче авто (CarId {updatedDriver.CarId})");
                throw new BadRequestException($"Неможливо оновити:
автомобіль з ID {updatedDriver.CarId} не існує.");
            }

            driver.FullName = updatedDriver.FullName;
```

```
            driver.Rating = updatedDriver.Rating;
            driver.IsBusy = updatedDriver.IsBusy;
            driver.CarId = updatedDriver.CarId;

            dataService.SaveData(data);
                logger.LogInformation($"[Успіх] Дані водія ID {id}
успішно оновлено.");
            return driver;
        }

        public void Delete(int id)
        {
            var data = dataService.LoadData();
              var driver = data.Driver.FirstOrDefault(d => d.Id ==
id);

            if (driver == null)
            {
                        logger.LogWarning($"[Увага] Спроба видалити
неіснуючого водія (ID {id})");
                    throw new NotFoundException($"Водія з ID {id} не
знайдено.");
            }

            data.Driver.Remove(driver);
            dataService.SaveData(data);
             logger.LogInformation($"[Успіх] Водія ID {id} видалено
з бази.");
        }

        public void DeleteAll()
        {
            var data = dataService.LoadData();
            data.Driver.Clear();
            dataService.SaveData(data);
```

```
            logger.LogInformation("[Увага] Всі водії були видалені
з бази даних!");
        }
    }
}
```

```
using System.Collections.Generic;
using Taxi.Models;

namespace Taxi.Services
{
    public interface ICarService
    {
        List<Car> GetAll();
        Car GetById(int id);
        Car Add(Car car);
        Car Update(int id, Car car);
        void Delete(int id);
        void DeleteAll();
        List<Driver> GetDriversByCarId(int carId);
    }
}
```

```
using System.Collections.Generic;
using System.Threading.Tasks;
using Taxi.Models;

namespace Taxi.Services
{
    public interface IDriverService
    {
        List<Driver> GetAll();
        Driver GetById(int id);
        Driver Add(Driver driver);
        Driver Update(int id, Driver driver);
        void Delete(int id);
```

```
        void DeleteAll();
    }
}
```

```
using System.Threading.Tasks;
using Taxi.Models;

namespace Taxi.Services
{
    public interface IJsonDataService
    {
        DataStore LoadData();
        void SaveData(DataStore data);
    }
}
```

```
using System.IO;
using System.Text.Json;
using Taxi.Models;

namespace Taxi.Services
{
    public class JsonDataService : IJsonDataService
    {
        private readonly string filePath = "data.json";

        public DataStore LoadData()
        {
            if (!File.Exists(filePath))
            {
                return new DataStore();
            }
            var json = File.ReadAllText(filePath);
              return JsonSerializer.Deserialize<DataStore>(json) ??
new DataStore();
        }
```

```
        public void SaveData(DataStore data)
        {
             var options = new JsonSerializerOptions { WriteIndented
= true };
            var json = JsonSerializer.Serialize(data, options);
            File.WriteAllText(filePath, json);
        }
    }
}
```

Program

```
using Taxi.Middleware;
using Taxi.Services;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddControllers();
//    Learn    more    about    configuring    Swagger/OpenAPI    at
https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddSingleton<IJsonDataService,
JsonDataService>();
builder.Services.AddScoped<ICarService, CarService>();
builder.Services.AddScoped<IDriverService, DriverService>();

var app = builder.Build();

app.UseMiddleware<ExceptionHandlingMiddleware>();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
```

```
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

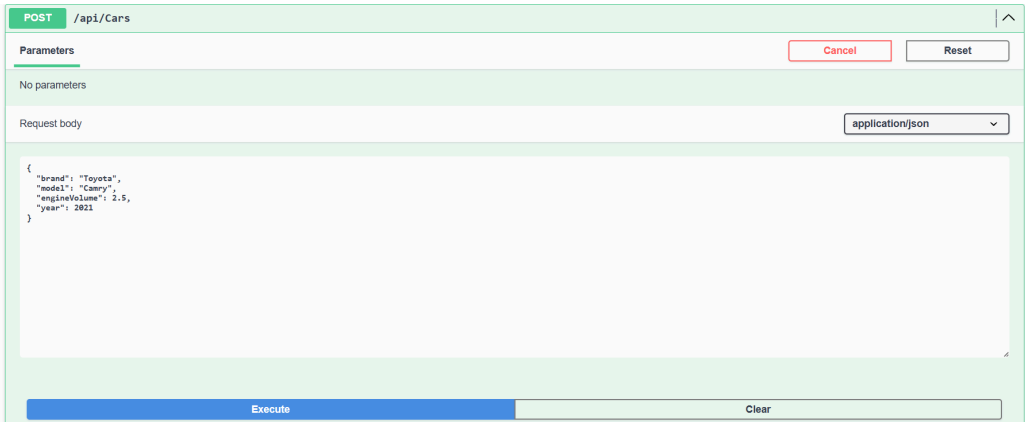
app.UseHttpsRedirection();

app.UseAuthorization();

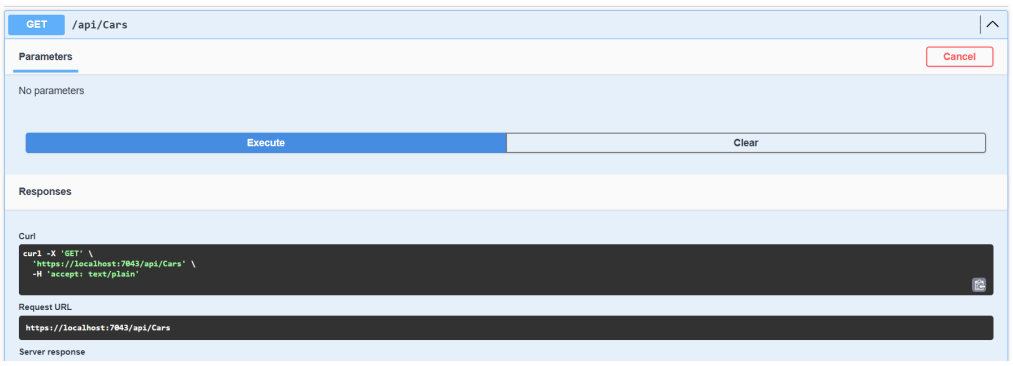
app.MapControllers();

app.Run();
```

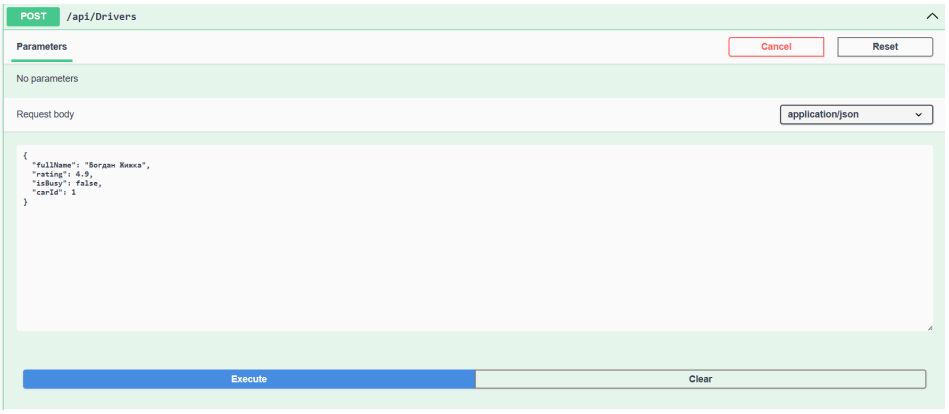
Знімки екрану з прикладами результатів роботи розробленої програми



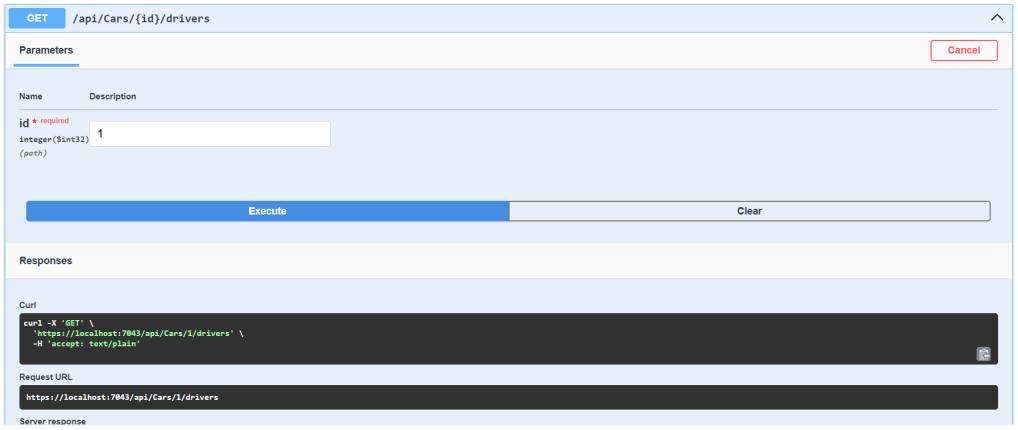


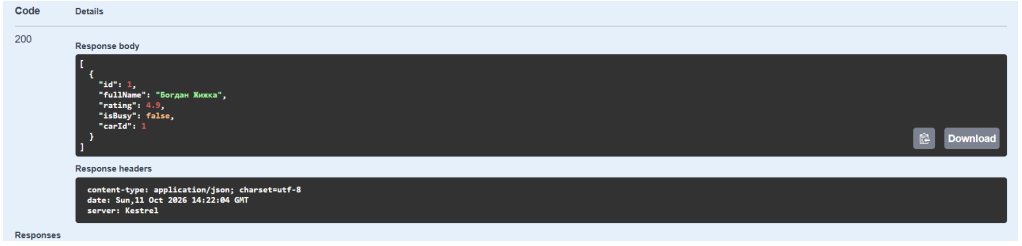


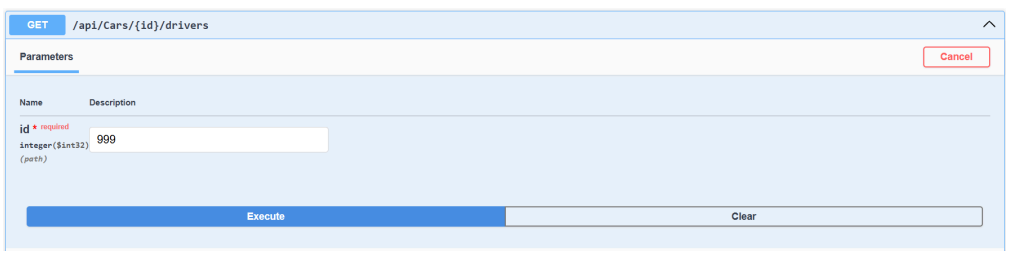

Code
Details
200
Response body
[
{
"id": 1,
"brand": "Toyota",
"model": "Camry",
"engineVolume": 2.5,
"year": 2021,
"age": 5
}
]
Download
Response headers
content-type: application/json; charset=utf-8
date: Sun,11 Oct 2026 14:18:32 GMT
server: Kestrel

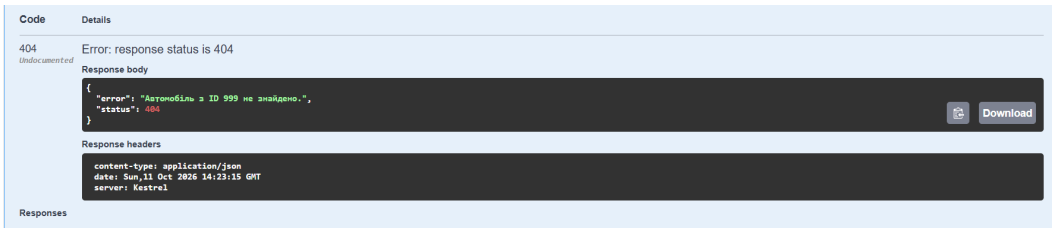

POST
/api/Drivers
Parameters
Cancel
Reset
No parameters
Request body
application/json
{
"fullName": "Богдан Хикка",
"rating": 4.9,
"isBusy": false,
"carId": 1
}
Execute
Clear

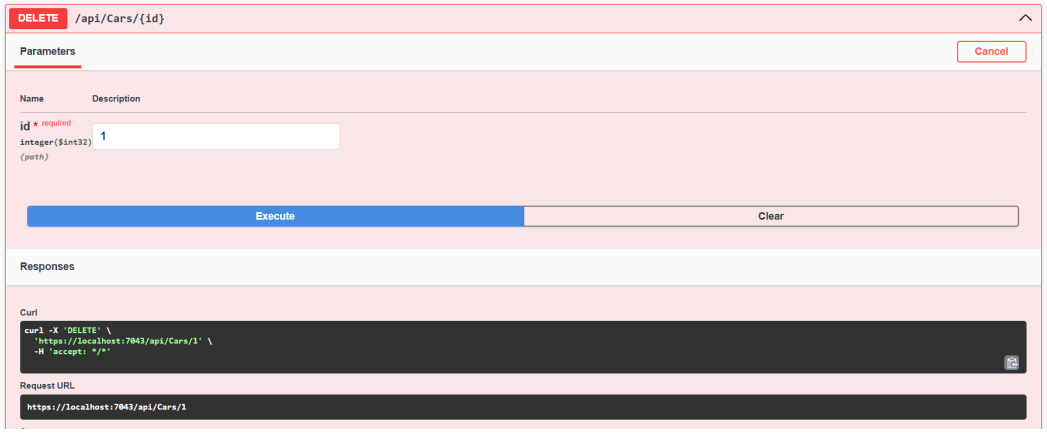

Code
Details
201
Undocumented
Response body
{
"id": 1,
"fullName": "Богдан Хикка",
"rating": 4.9,
"isBusy": false,
"carId": 1
}
Download
Response headers
content-type: application/json; charset=utf-8
date: Sun,11 Oct 2026 14:20:38 GMT
location: https://localhost:7043/api/Drivers/1
server: Kestrel


GET
/api/Cars/{id}/drivers
Parameters
Cancel
Name
Description
id * required
integer($int32)
1
(path)
Execute
Clear
Responses
Curl
curl -X 'GET' \
'https://localhost:7043/api/Cars/1/drivers' \
-H 'accept: text/plain'
Request URL
https://localhost:7043/api/Cars/1/drivers
Server response


Code
Details
200
Response body
[
{
"id": 1,
"fullName": "Богдан Хикка",
"rating": 4.9,
"isBusy": false,
"carId": 1
}
]
Download
Response headers
content-type: application/json; charset=utf-8
date: Sun,11 Oct 2026 14:22:04 GMT
server: Kestrel
Responses


GET
/api/Cars/{id}/drivers
Parameters
Cancel
Name
Description
id * required
integer($int32)
999
(path)
Execute
Clear








```
D:/Desktop/Lab_Web/Taxi/Taxi/bin/Debug/net8.0/Taxi.exe
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: https://localhost:7043
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5257
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: D:\Desktop\Lab_Web\Taxi\Taxi
info: Taxi.Services.CarService[0]
      [Усп?х] Додано новий автомоб?ль: Toyota Camry, ID: 1
info: Taxi.Services.CarService[0]
      Виконано запит на отримання вс?х автомоб?л?в.
info: Taxi.Services.CarService[0]
      Виконано запит на отримання вс?х автомоб?л?в.
info: Taxi.Services.DriverService[0]
      [Усп?х] Додано нового вод?я: Богдан Жижка, ID: 1
warn: Taxi.Services.CarService[0]
      [Увага] Спроба знайти вод?їв для не?снуючого авто (ID 999)
info: Taxi.Services.CarService[0]
      [Усп?х] Автомоб?ль ID 1 та його вод?їв видалено з бази.
info: Taxi.Services.CarService[0]
      Виконано запит на отримання вс?х автомоб?л?в.
```

Висновок

Під час виконання комп'ютерного практикуму було спроєктовано та успішно реалізовано REST API для системи управління таксопарком. Я розробив дві пов'язані сутності (Автомобіль та Водій) і реалізував для них усі базові CRUD-операції, включно з отриманням водіїв для конкретного авто. Додатково було налаштовано збереження даних у JSON-файл, логування роботи програми та власну глобальну обробку помилок (Middleware).